

LAMBDA EXPRESSIONS AND INTRODUCTION TO RECURSION

COMPUTER SCIENCE MENTORS 61A

September 14, 2026 – September 18, 2026

1 Lambda Expressions

1. Draw the environment diagram that results from running the code.

```
def f(g):  
    k = 2  
    h = lambda x: g(x)  
    apply = lambda fn: fn(4)  
    return apply(h)  
k = 7  
f(lambda x: x + k - 3)
```

2. What would Python display?

- (a) `(lambda f: f(f))(lambda x: 6)`
(b) `(lambda f, g: g(f))(add, lambda op: op(2, 7))`

3. Make a lambda function, `make_multiple_checker()`, that takes in an integer `divisor`, and returns a function that takes in a value `x` and checks whether `x` is a multiple of `divisor`.

```
>>> make_multiple_checker = _____  
>>> multiple_of_4 = make_multiple_checker(4)  
>>> multiple_of_4(20)  
True  
>>> multiple_of_4(18)  
False
```

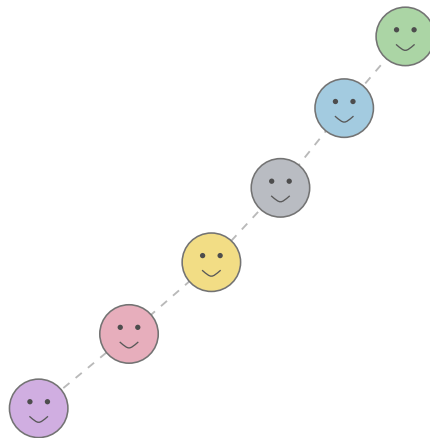
2 Introduction to Recursion

There are three steps to writing a recursive function:

1. Create base case(s)
2. Reduce your problem to a smaller subproblem and call your function recursively to solve the smaller subproblem(s)
3. Use the subproblems' solutions as pieces to construct a larger problem's solution (This can happen in many layers!)

Real World Analogy for Recursion

Imagine that you're waiting in a long line for a campus shuttle and you want to know what position you're in.



You decide to ask the person in front of you how many people are in front of them. That way, you can take their response and add 1 to it to find your place. Now, the person in front of you is faced with the same problem that you were trying to solve, with one less person in front of them than you. They use the same idea and ask the person in front of them. This continues until the first person in line is asked. The first person knows there are 0 people in front of them, so that answer can be passed backward through the line until it reaches you.

Looking at this example, we broke the question “how many people are in front of me?” into 1 + “how many people are in front of the person ahead of me?” The process stops at the first person in line, where the answer is immediately known. In more formal terms, the smaller repeated question gives us a **recurrence relationship**, and the stopping case is called the **base case**.

As a program goes through recursion, it does not finish the original problem until it reaches a base case. It then works its way back from the base case to the original input to build the final answer. Just like your question as you stand in line, the program:

1. goes out (down the call stack), person by person (call by call) (**recursive case**)
 2. until the person at the very front is reached (**base case**)
 3. when the answers get combined and eventually passed back to you, solving the problem. (**constructing the final solution with solutions to subparts**)
4. Translate the line example into Python! We can represent our shuttle line as an integer of 0s and 2s, where the 0s represent an empty space and the 2s represent a person standing in line. You are about to stand at the end of the line, but first, you need to count how many people would be in front of you! Complete the definition for `shuttle_line`, which takes in an integer `n` and returns the number of 2s in it. Implement the function recursively, with a base case and a recursive case.

Hint: To get the last digit of an integer `n`, you can do `n % 10`. To remove the last digit from an integer, you can do `n // 10`.

```
def shuttle_line(n):
    """Takes in an integer of 0s and 2s and returns the number of 2s.

    >>> shuttle_line(0)
    0
    >>> shuttle_line(2)
    1
    >>> shuttle_line(20202)
    3
    >>> shuttle_line(20002)
    2
    """
    if _____ :                # base case

        return _____

    else:

        return _____ + _____ # recursive case
```

5. The *Tibonacci sequence* is a recursively defined sequence of integers; we denote the n th Tibonacci number t_n . The first three terms of the sequence are $t_0 = 1$, $t_1 = 2$, $t_2 = 4$. For $n \geq 3$, t_n is defined as the sum of the previous three terms in the sequence.

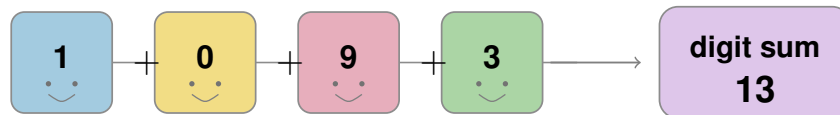
Complete the function `tib`, which takes in an integer `n` and returns the n th Tibonacci number, t_n . Also, identify the three parts of recursive function design as they are used in your solution.

```
def tib(n):  
    """  
    >>> tib(0)  
    1  
    >>> tib(1)  
    2  
    >>> tib(2)  
    4  
    >>> tib(3) # tib(2) + tib(1) + tib(0) = 7  
    7  
    >>> tib(4) # tib(3) + tib(2) + tib(1) = 13  
    13  
    """  
    if _____:  
        return _____  
    return _____
```

6. Including the original call, how many calls are made to `tib` when you evaluate `tib(5)`?

7. Complete the definition for `sum_digits`, which takes in a nonnegative integer `n` and returns the sum of its digits. Implement the function recursively.

```
def sum_digits(n):  
    """Takes in a nonnegative integer and returns the sum of its  
    digits.  
  
    >>> sum_digits(0)  
    0  
    >>> sum_digits(5)  
    5  
    >>> sum_digits(71)  
    8  
    >>> sum_digits(1093)  
    13  
    """
```



A tiny digit parade: solve the smaller number, then add the last digit.

8. Implement a recursive version of popzap.

```
def popzap(n):  
    """Prints the numbers from 1 to n. If the number is divisible by 4, it  
    instead prints 'pop'. If the number is divisible by 6, it instead  
    prints 'zap'. If the number is divisible by both, it prints 'popzap'.  
    You must do this recursively!  
  
    >>> popzap(12)  
    1  
    2  
    3  
    pop  
    5  
    zap  
    7  
    pop  
    9  
    10  
    11  
    popzap  
    """
```

LAMBDA EXPRESSIONS AND INTRODUCTION TO RECURSION

Solutions

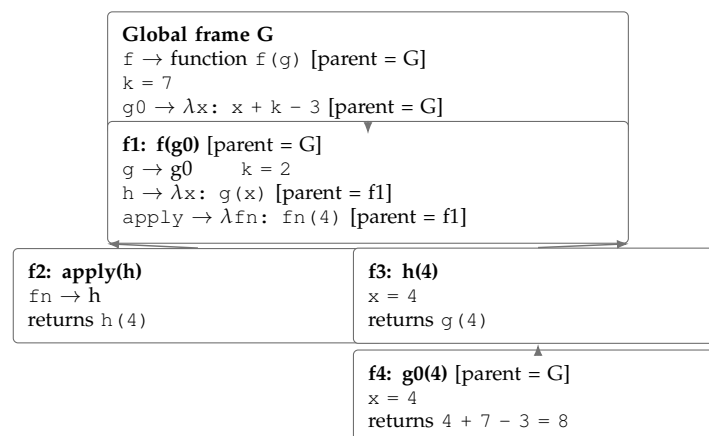
COMPUTER SCIENCE MENTORS 61A

September 14, 2026 – September 18, 2026

1 Lambda Expressions

1. Draw the environment diagram that results from running the code.

```
def f(g):  
    k = 2  
    h = lambda x: g(x)  
    apply = lambda fn: fn(4)  
    return apply(h)  
  
k = 7  
f(lambda x: x + k - 3)
```



The final value is 8. The lambda passed into f was created in the global frame, so when it evaluates k, it uses the global k = 7, not the local k = 2.

2. What would Python display?

- (a) (lambda f: f(f))(lambda x: 6) 6
- (b) (lambda f, g: g(f))(add, lambda op: op(2, 7)) 9

3. Make a lambda function, `make_multiple_checker()`, that takes in an integer `divisor`, and returns a function that takes in a value `x` and checks whether `x` is a multiple of `divisor`.

```
>>> make_multiple_checker = _____
>>> multiple_of_4 = make_multiple_checker(4)
>>> multiple_of_4(20)
True
>>> multiple_of_4(18)
False

>> make_multiple_checker = lambda divisor: lambda x: x % divisor == 0
>> multiple_of_4 = make_multiple_checker(4)
>> multiple_of_4(20)
True
>> multiple_of_4(18)
False
```

The outer lambda “remembers” `divisor`. After `make_multiple_checker(4)`, the returned function checks whether its input has remainder 0 when divided by 4.

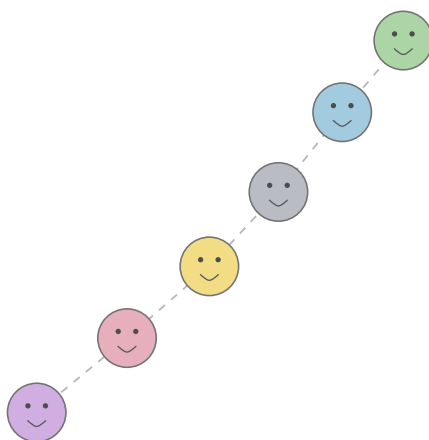
2 Introduction to Recursion

There are three steps to writing a recursive function:

1. Create base case(s)
2. Reduce your problem to a smaller subproblem and call your function recursively to solve the smaller subproblem(s)
3. Use the subproblems' solutions as pieces to construct a larger problem's solution

Real World Analogy for Recursion

Imagine that you're waiting in a long line for a campus shuttle and you want to know what position you're in.



Each person asks the person ahead the same smaller question. Eventually the question reaches the first person, where the answer is immediate: 0 people are in front. That is the **base case**. On the way back, each person adds 1, which constructs the solution to the larger problem.

When solving
recursive call

4. Complete `shuttle_line`, which takes an integer made of 0s and 2s and recursively returns the number of 2s.

```
def shuttle_line(n):  
    """Takes in an integer of 0s and 2s and returns the number of 2s.  
  
    >>> shuttle_line(0)  
    0  
    >>> shuttle_line(2)  
    1  
    >>> shuttle_line(20202)  
    3  
    >>> shuttle_line(20002)  
    2  
    """  
    if _____ :                # base case  
        return _____  
    else:  
        return _____ + _____ # recursive case  
  
if n == 0:  
    return 0  
else:  
    return (n % 10 == 2) + shuttle_line(n // 10)
```

The recursive call removes one digit using `n // 10`. The expression `n % 10 == 2` contributes 1 when the last digit is 2 and 0 otherwise. The base case `n == 0` stops the recursion.

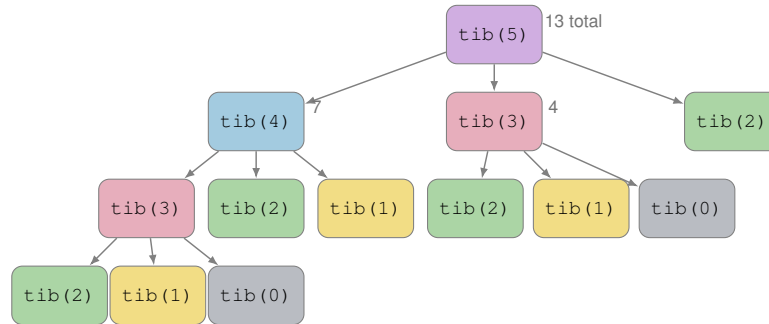
5. The *Tibonacci sequence* begins $t_0 = 1$, $t_1 = 2$, $t_2 = 4$, and for $n \geq 3$, t_n is the sum of the previous three terms. Complete `tib(n)` and identify the three parts of recursive design.

```
def tib(n):  
    """  
    >>> tib(0)  
    1  
    >>> tib(1)  
    2  
    >>> tib(2)  
    4  
    >>> tib(3)  
    7  
    >>> tib(4)  
    13  
    """  
    if _____:  
        return _____  
    return _____  
  
if n <= 2:  
    return 2 ** n  
return tib(n - 1) + tib(n - 2) + tib(n - 3)
```

- Base cases: $n \leq 2$, returning 1, 2, 4 as $2 ** n$.
- Smaller subproblems: `tib(n - 1)`, `tib(n - 2)`, `tib(n - 3)`.
- Construct the larger answer by adding the three recursive results.

6. Including the original call, how many calls are made to `tib` when you evaluate `tib(5)`? **13 calls.**

One way to see it is to draw the full call tree. This tree is newly drawn for this worksheet; every non-base call has exactly three children.



$$C(0) = C(1) = C(2) = 1$$

$$C(3) = 1 + C(2) + C(1) + C(0) = 4$$

$$C(4) = 1 + C(3) + C(2) + C(1) = 7$$

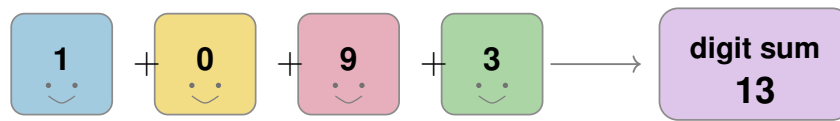
$$C(5) = 1 + C(4) + C(3) + C(2) = 13$$

The recurrence for the number of calls is the same shape as the function itself, except each call also counts the current call with the leading 1.

7. Complete `sum_digits`, which takes in a nonnegative integer `n` and recursively returns the sum of its digits.

```
def sum_digits(n):  
    """Takes in a nonnegative integer and returns the sum of its digits.  
  
    >>> sum_digits(0)  
    0  
    >>> sum_digits(5)  
    5  
    >>> sum_digits(71)  
    8  
    >>> sum_digits(1093)  
    13  
    """  
  
    if n < 10:  
        return n  
    return n % 10 + sum_digits(n // 10)
```

A one-digit number is already its own digit sum. Otherwise, add the last digit to the digit sum of the remaining prefix.



A tiny digit parade: peel off one digit at a time.

8. Implement a recursive version of popzap.

```
def popzap(n):  
    """Prints the numbers from 1 to n. Multiples of 4 print 'pop',  
    multiples of 6 print 'zap', and multiples of both print 'popzap'.  
    You must do this recursively!  
    """  
  
    if n == 0:  
        return  
    popzap(n - 1)  
    if n % 12 == 0:  
        print('popzap')  
    elif n % 4 == 0:  
        print('pop')  
    elif n % 6 == 0:  
        print('zap')  
    else:  
        print(n)
```

The recursive call comes *before* the printing logic, so the smaller call prints 1 through $n - 1$ first. Then the current call prints the correct representation of n . Checking 12 first handles numbers divisible by both 4 and 6.



The recursion does the counting; the current call chooses the word.